

Assignment 5: Data Visualization

Student Name

Fall 2025

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on Data Visualization

Directions

1. Rename this file `<FirstLast>_A05_DataVisualization.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure your code is tidy; use line breaks to ensure your code fits in the knitted output.
5. Be sure to **answer the questions** in this assignment document.
6. When you have completed the assignment, **Knit** the text and code into a single PDF file.

Set up your session

1. Set up your session. Load the tidyverse, here & cowplot packages, and verify your home directory. Read in the NTL-LTER processed data files for nutrients and chemistry/physics for Peter and Paul Lakes (use the tidy NTL-LTER_Lake_Chemistry_Nutrients_PeterPaul_Processed.csv version in the Processed_KEY folder) and the processed data file for the Niwot Ridge litter dataset (use the NEON_NIWO_Litter_mass_trap_Processed.csv version, again from the Processed_KEY folder).
2. Make sure R is reading dates as date format; if not change the format to date.

```
#1
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    4.0.0      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.1.0
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(ggplot2)
library(here)
```

```
## here() starts at /Users/kaitlynkukula/EDE_Fall2025
```

```
library(cowplot)
```

```
##
## Attaching package: 'cowplot'
##
## The following object is masked from 'package:lubridate':
##
##      stamp
```

```
getwd()
```

```
## [1] "/Users/kaitlynkukula/EDE_Fall2025"
```

```
peterpaul_nutrients <- read.csv(here("Data/Processed_KEY/NTL-LTER_Lake_Chemistry_Nutrients_PeterPaul_Pr"),
                               stringsAsFactors = TRUE)
niwot_litter <- read.csv(here("Data/Processed_KEY/NEON_NIWO_Litter_mass_trap_Processed.csv"),
                        stringsAsFactors = TRUE)
```

```
#2: Converting factors to dates
class(peterpaul_nutrients$sampledte)
```

```
## [1] "factor"
```

```
peterpaul_nutrients$sampledte <- ymd(peterpaul_nutrients$sampledte)
```

```
# Convert month to factor
peterpaul_nutrients$month <- factor(peterpaul_nutrients$month, levels = c(1:12), labels = month.abb)
class(niwot_litter$collectDate)
```

```
## [1] "factor"
```

```
niwot_litter$collectDate <- ymd(niwot_litter$collectDate)
```

Define your theme

3. Build a theme and set it as your default theme. Customize the look of at least two of the following:

- Plot background
- Plot title
- Axis labels
- Axis ticks/gridlines
- Legend

#3: Generate a unique theme

```
kak_theme <- theme_bw() +  
  theme(plot.title = element_text(size = 11, hjust = 0.5, face = "bold"), #Adjust title  
        axis.text.x = element_text(size = 9), # Adjust x-axis values  
        axis.text.y = element_text(size = 9), # Adjust y-axis values  
        axis.title.x = element_text(size = 10.5, face = "bold"), # Adjust x-axis title  
        axis.title.y = element_text(size = 10.5, face = "bold"), # Adjust y-axis title  
        legend.position = "bottom", # Define legend position  
        legend.text = element_text(size = 9), # Define legend entry sizes  
        legend.title = element_text(size = 9)) # Define legend name sizes
```

Create graphs

For numbers 4-7, create ggplot graphs and adjust aesthetics to follow best practices for data visualization. Ensure your theme, color palettes, axes, and additional aesthetics are edited accordingly.

4. [NTL-LTER] Plot total phosphorus (tp_ug) by phosphate (po4), with separate aesthetics for Peter and Paul lakes. Add line(s) of best fit using the `lm` method. Adjust your axes to hide extreme values (hint: change the limits using `xlim()` and/or `yylim()`).

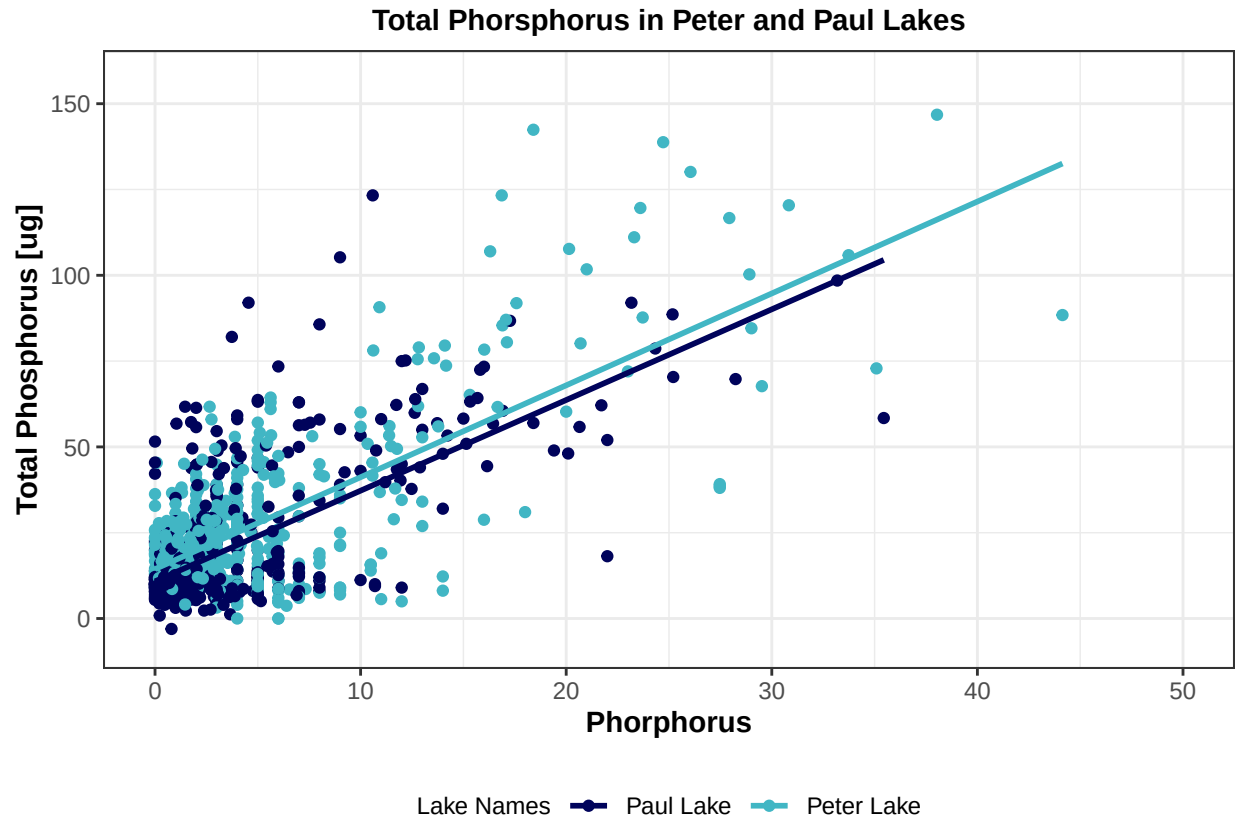
#4: total phosphorus vs phosphate

```
peterpaul_phosphorusbylake <- ggplot(peterpaul_nutrients,  
                                     aes(x = po4, y = tp_ug, color = lakename)) +  
  geom_point() +  
  geom_smooth(method = lm, se = FALSE) +  
  xlim(0, 50) +  
  
  scale_color_manual(values = c("#00035B", "#41b6c4")) +  
  labs(x = "Phosphorus", y = "Total Phosphorus [ug]",  
       title = "Total Phosphorus in Peter and Paul Lakes",  
       color = "Lake Names") +  
  kak_theme  
print(peterpaul_phosphorusbylake)
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

```
## Warning: Removed 21947 rows containing non-finite outside the scale range  
## ('stat_smooth()').
```

```
## Warning: Removed 21947 rows containing missing values or values outside the scale range  
## ('geom_point()').
```



5. [NTL-LTER] Make three separate boxplots of (a) temperature, (b) TP, and (c) TN, with month as the x axis and lake as a color aesthetic. Then, create a cowplot that combines the three graphs. Make sure that only one legend is present and that graph axes are aligned.

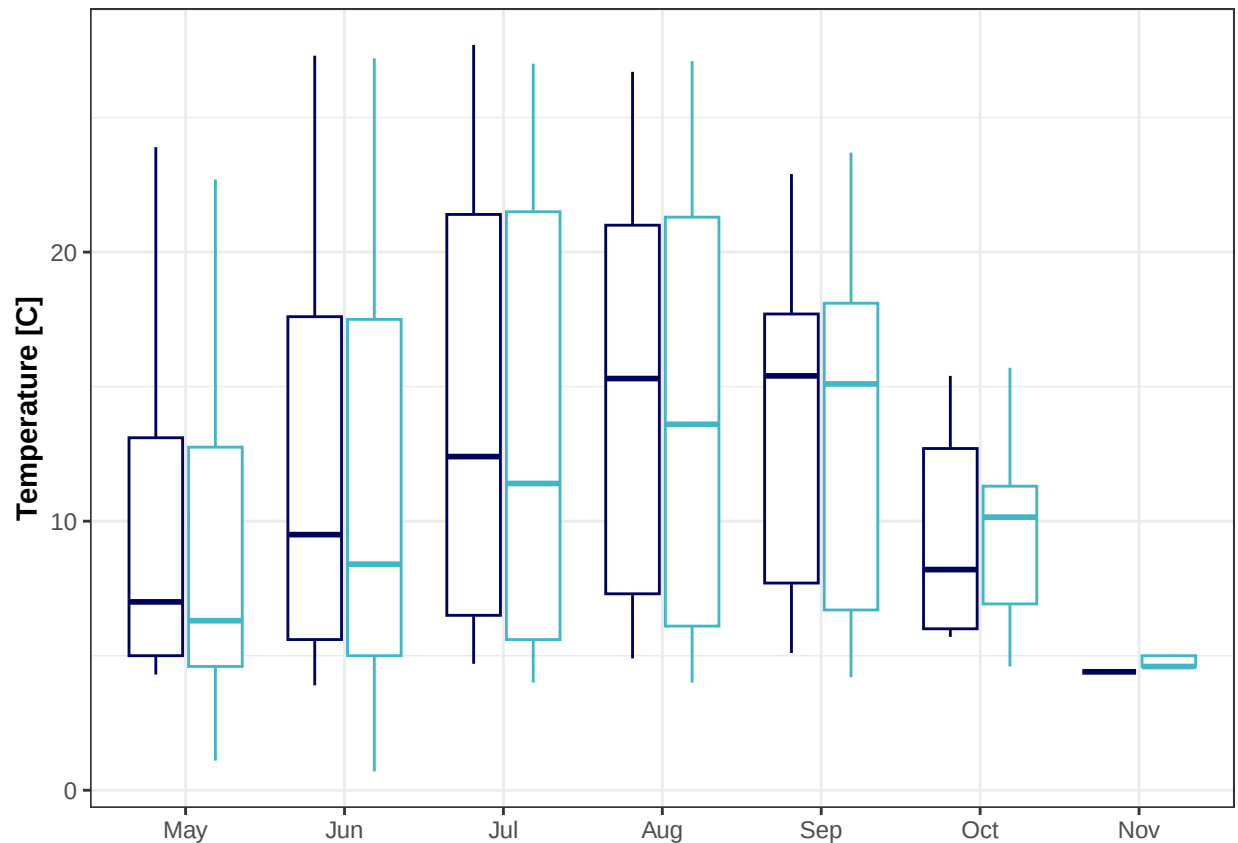
Tips: * Recall the discussion on factors in the lab section as it may be helpful here. * Setting an axis title in your theme to `element_blank()` removes the axis title (useful when multiple, aligned plots use the same axis values) * Setting a legend's position to "none" will remove the legend from a plot. * Individual plots can have different sizes when combined using `cowplot`.

```
#5

# Create a plot for temperature
peterpaul_temperature <- ggplot(peterpaul_nutrients %>% filter(month != "Feb")) +
  geom_boxplot(aes(x = month, y = temperature_C, color = lakename)) +
  labs(x = element_blank(), y = "Temperature [C]") +
  scale_color_manual(values = c("#00035B", "#41b6c4")) +
  kak_theme +
  theme(legend.position = "none")
print(peterpaul_temperature)
```

```
## Warning: Removed 3550 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```

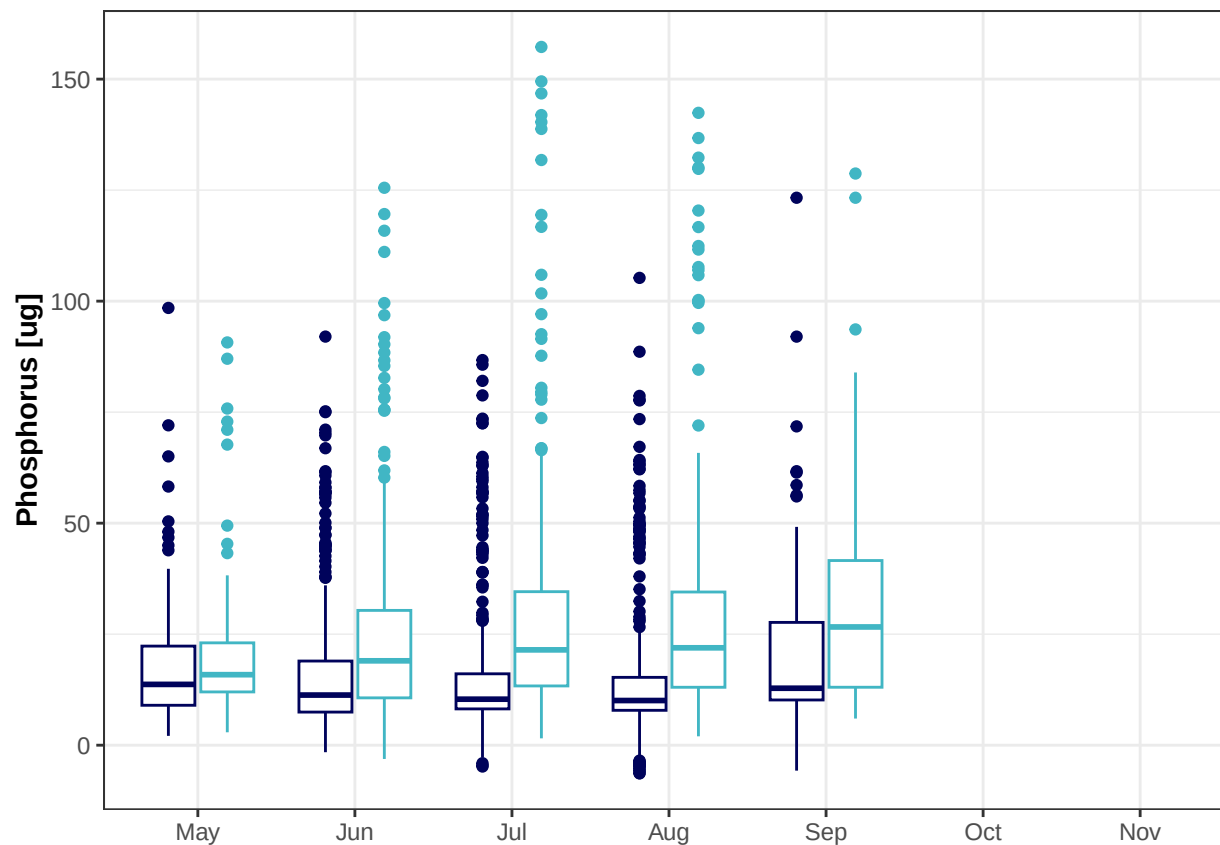
```
## Warning: 'label' cannot be a <ggplot2::element_blank> object.
```



Create a plot for total phosphorus

```
peterpaul_phosphorus <- ggplot(peterpaul_nutrients %>% filter(month != "Feb")) +
  geom_boxplot(aes(x = month, y = tp_ug, color = lakename)) +
  labs(x = element_blank(), y = "Phosphorus [ug]") +
  scale_color_manual(values = c("#00035B", "#41b6c4")) +
  kak_theme +
  theme(legend.position = "none")
print(peterpaul_phosphorus)
```

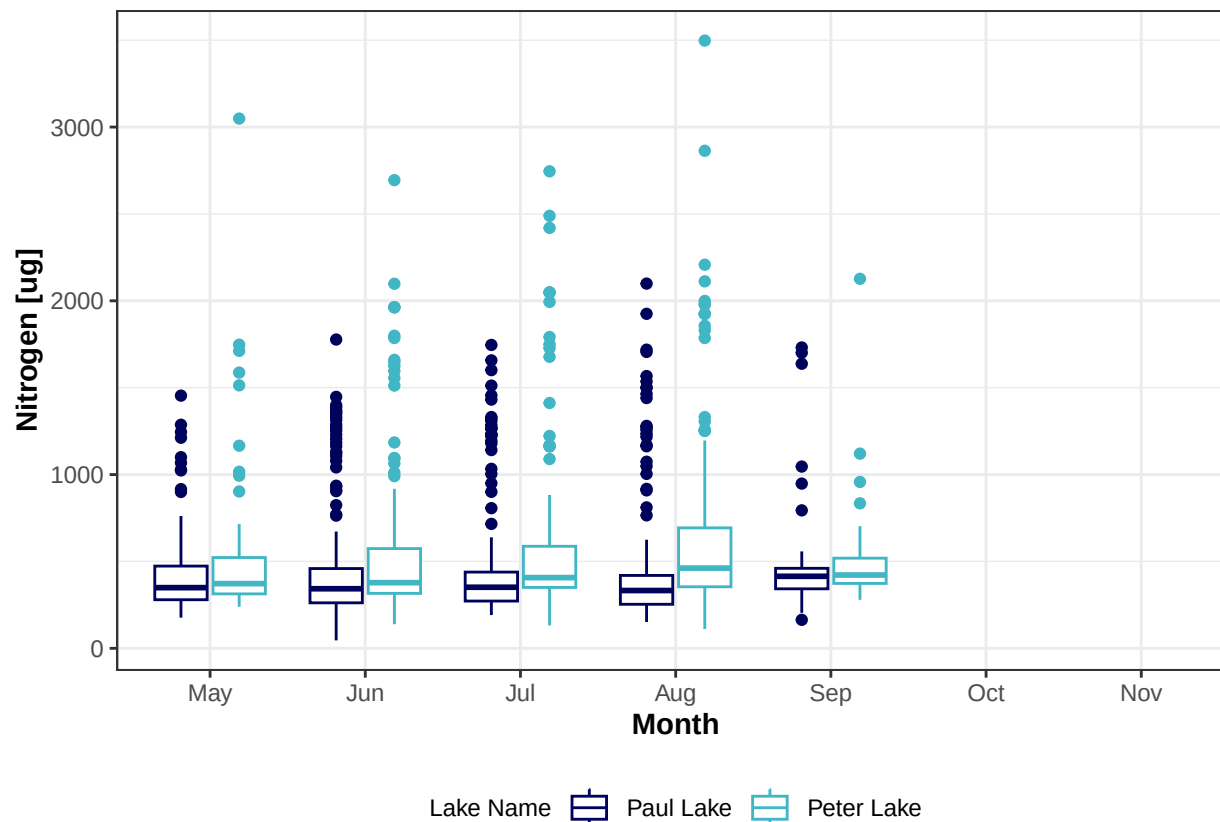
```
## Warning: Removed 20713 rows containing non-finite outside the scale range
## ('stat_boxplot()').
## 'label' cannot be a <ggplot2::element_blank> object.
```



Create a plot for total nitrogen

```
peterpaul_nitrogen <- ggplot(peterpaul_nutrients %>% filter(month != "Feb")) +
  geom_boxplot(aes(x = month, y = tn_ug, color = lakename)) +
  labs(x = "Month", y = "Nitrogen [ug]",
       color = "Lake Name") +
  scale_color_manual(values = c("#00035B", "#41b6c4")) +
  kak_theme
print(peterpaul_nitrogen)
```

```
## Warning: Removed 21567 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```



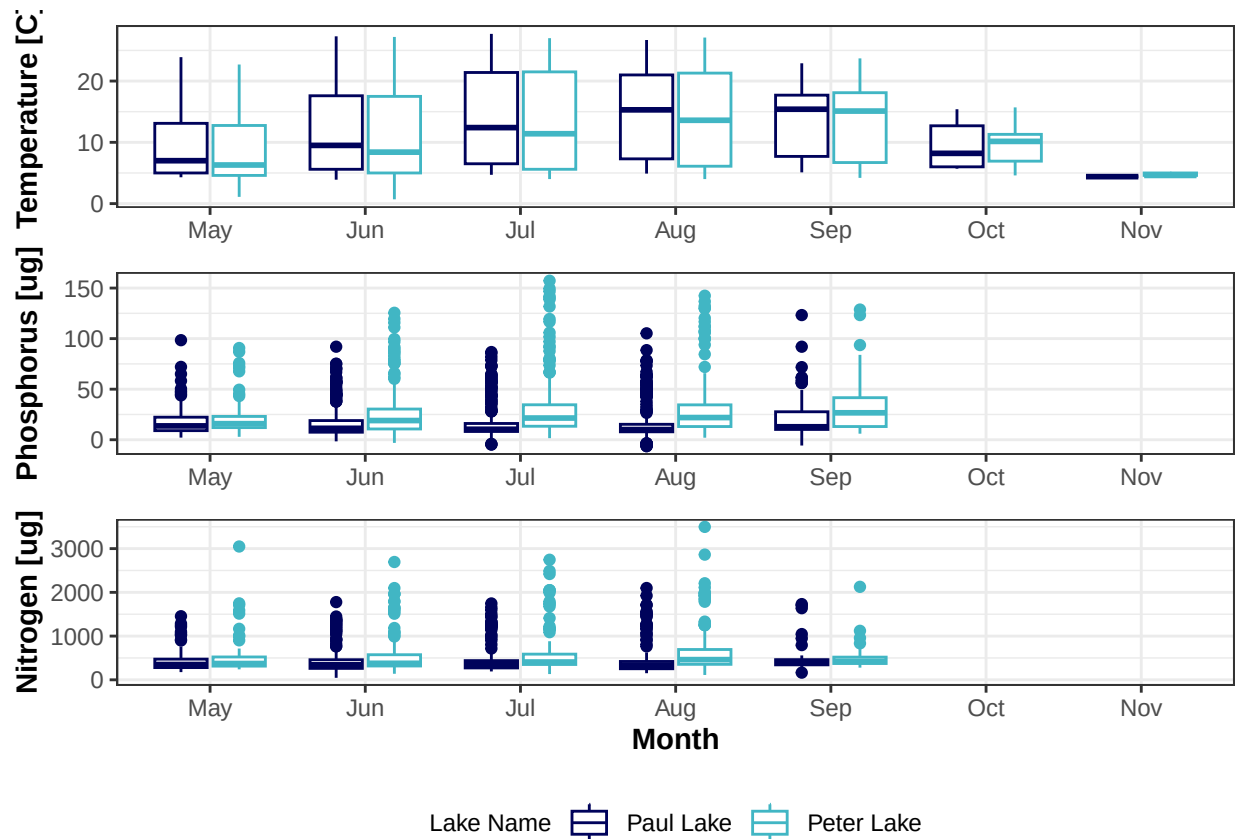
```
plot_grid(peterpaul_temperature, peterpaul_phosphorus, peterpaul_nitrogen,
  ncol = 1,
  align = "v",
  rel_heights = c(1,1,1.5))
```

```
## Warning: Removed 3550 rows containing non-finite outside the scale range
## ('stat_boxplot()').
## 'label' cannot be a <ggplot2::element_blank> object.
```

```
## Warning: Removed 20713 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```

```
## Warning: 'label' cannot be a <ggplot2::element_blank> object.
```

```
## Warning: Removed 21567 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```



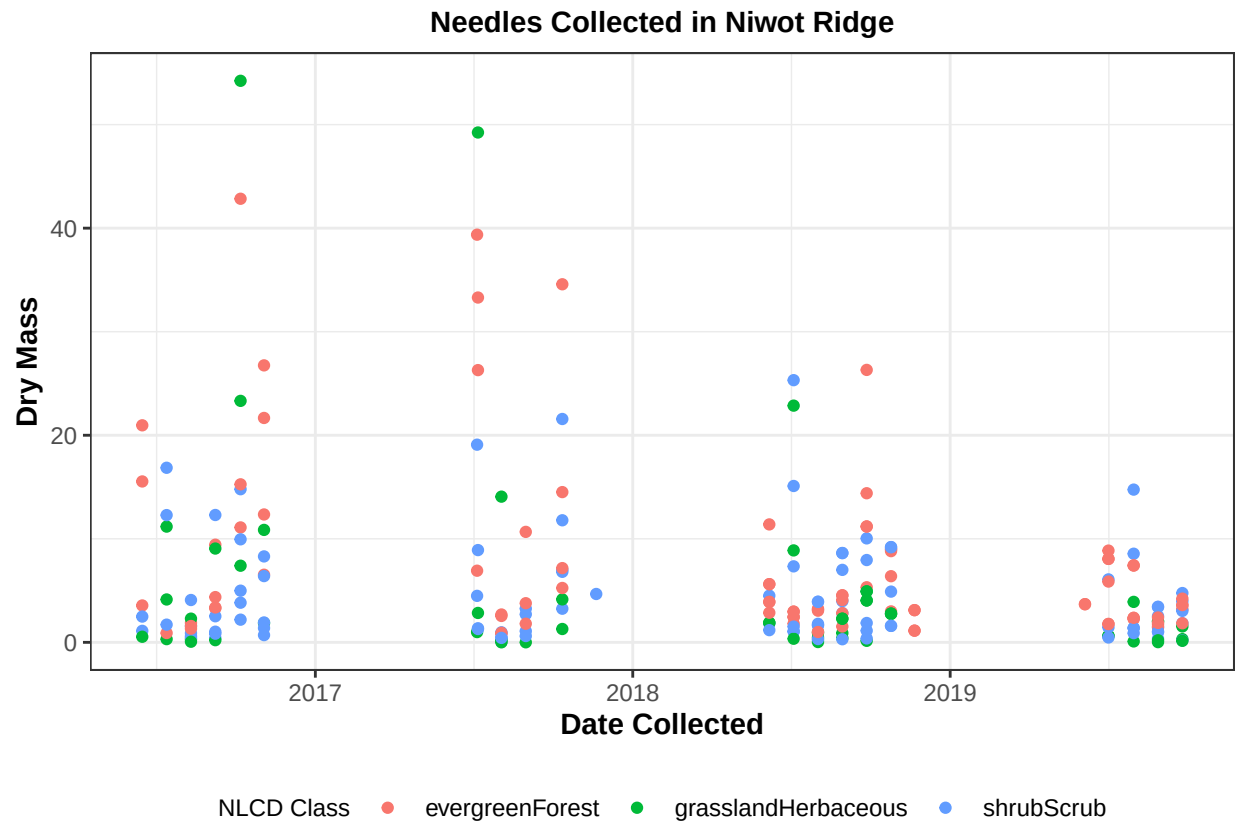
Question: What do you observe about the variables of interest over seasons and between lakes?

Answer: Temperature is the most variable observation, with temperatures increasing through the summer months and decreasing during the winter. Peter Lake has slightly more phosphorus each month compared to Paul Lake, but the differences may not be significant. Similar sentiments can be said about nitrogen, where Peter Lake has slightly higher values for each month but the differences are small.

6. [Niwot Ridge] Plot a subset of the litter dataset by displaying only the “Needles” functional group. Plot the dry mass of needle litter by date and separate by NLCD class with a color aesthetic. (no need to adjust the name of each land use)
7. [Niwot Ridge] Now, plot the same plot but with NLCD classes separated into three facets rather than separated by color.

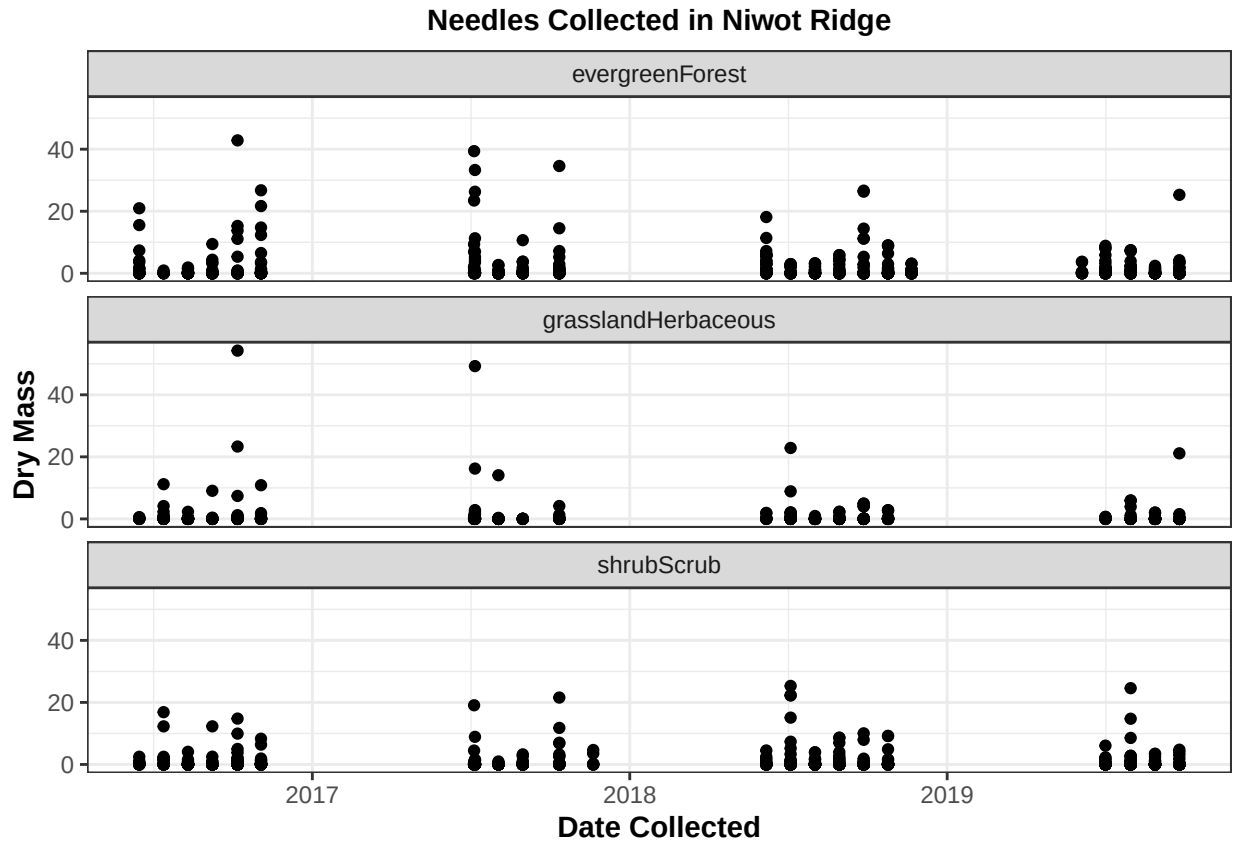
*#6: Create a plot looking at needles only, comparing dry mass collected based on date using colors
differentiate NLCD classes*

```
niwot_needles_colors <- ggplot(niwot_litter %>% filter(functionalGroup == "Needles")) +
  geom_point(aes(x = collectDate, y = dryMass, color = nlcdClass)) +
  labs(x = "Date Collected", y = "Dry Mass",
       title = "Needles Collected in Niwot Ridge",
       color = "NLCD Class") +
  kak_theme
print(niwot_needles_colors)
```

#7: Create a plot with the same inputs as question 6, but using facets to show different classes

```
niwot_needles_facets <- ggplot(niwot_litter) +
  geom_point(aes(x = collectDate, y = dryMass)) +
  facet_wrap(vars(nlcdClass), ncol = 1) +
  labs(x = "Date Collected", y = "Dry Mass",
       title = "Needles Collected in Niwot Ridge") +
  kak_theme
print(niwot_needles_facets)
```



Question: Which of these plots (6 vs. 7) do you think is more effective, and why?

Answer: I personally find that using facets is a more effective way to display the information as it is easier to quickly see trends between different NLCD classes and dry mass collected, as well as trends in each NLCD class per year. Using the sorting by colors method does not reflect